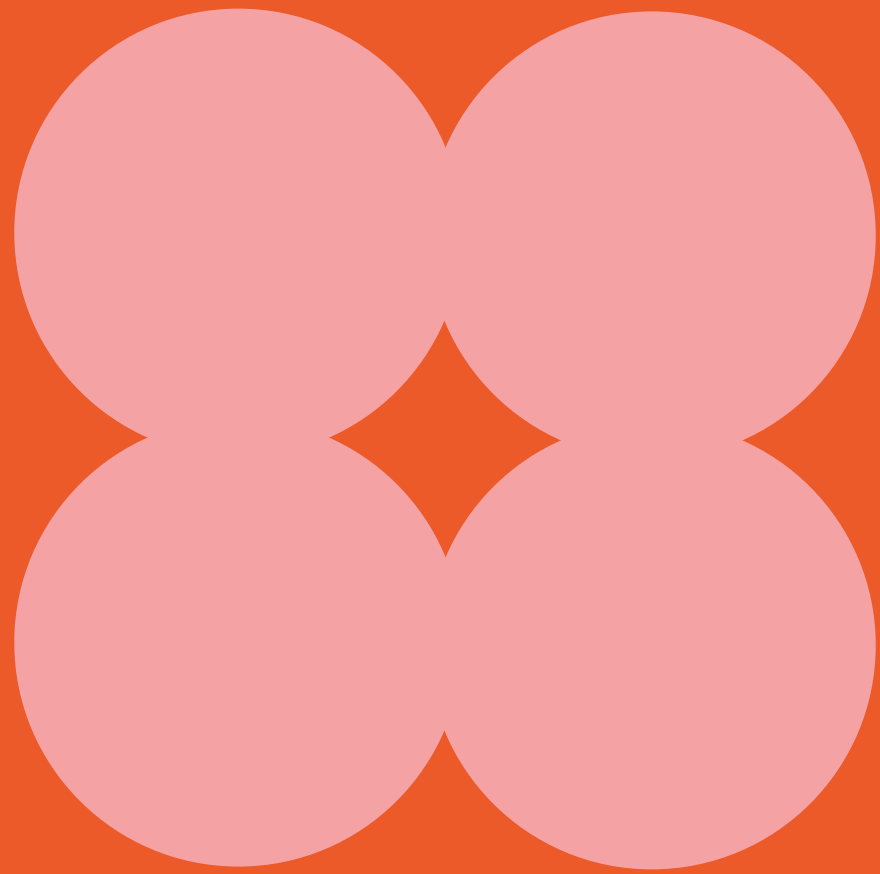




22/01/2026 Tarun sharma

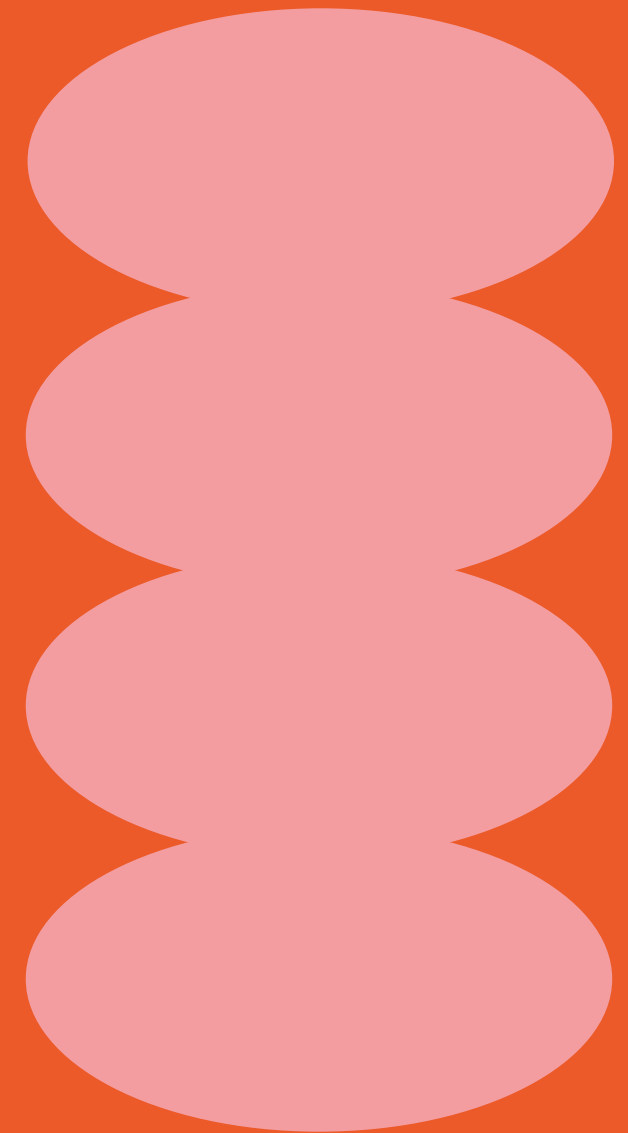
MASTERING DATABASES IN NODE.JS

Prisma ORM – Modern, Type-Safe
Database Access



AGENDA

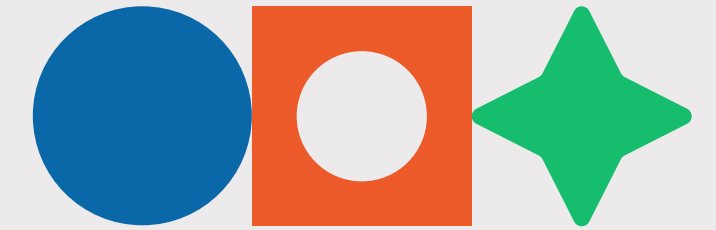
1. Introduction to Prisma
2. Setup and Project Structure
3. Schema and Models
4. Migrations and Client Usage
5. CRUD Operations
6. Advanced Features
7. Tools and Best Practices
8. Use Cases and Comparisons





INTRODUCTION TO PRISMA

WHY PRISMA?



TypeScript-First, Modern ORM

Designed for seamless TypeScript integration

End-to-End Type Safety

Compile-time safety for all database operations

Exceptional Developer Experience

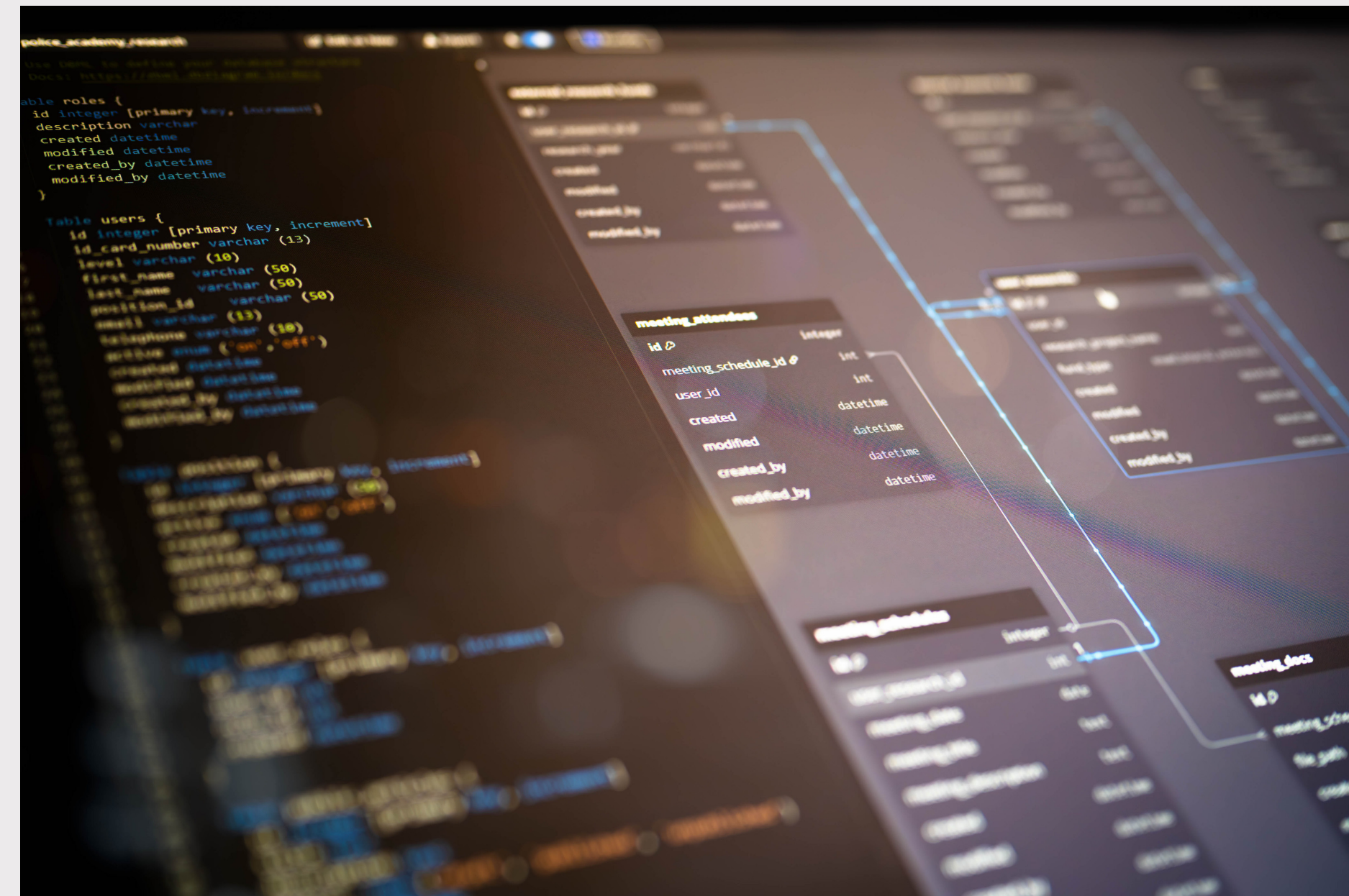
Intuitive API and rapid feedback for productivity

Effortless Node.js & NestJS Integration

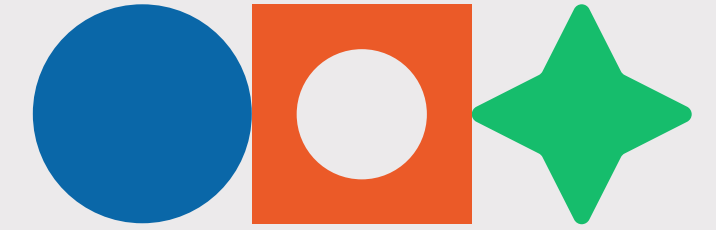
Works out-of-the-box with popular frameworks

Trusted by Startups & SaaS

Widely adopted for modern, scalable applications



DATABASES SUPPORTED BY PRISMA



Broad Database Compatibility

Supports PostgreSQL, MySQL, and SQLite out of the box

Enterprise-Ready Options

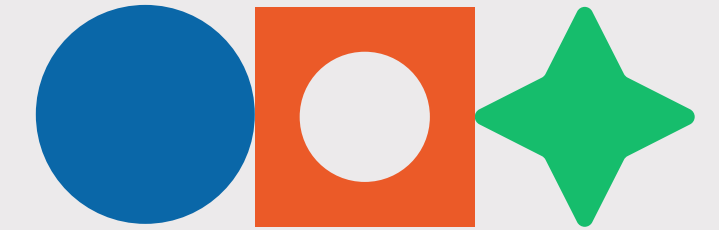
Works with SQL Server for large-scale applications

NoSQL Support

Integrates with MongoDB for flexible data models



PRISMA ARCHITECTURE



Prisma Schema: Single Source of Truth

Centralizes models, relations, and config in one file

Type-Safe Prisma Client Generation

Auto-generates a strongly-typed API for database access

Optimized Query Engine

Executes performant queries tailored to your schema

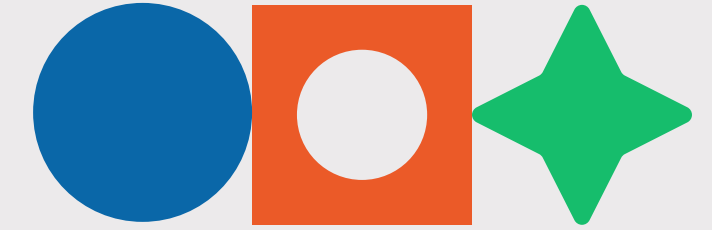
Prisma Studio & Migrations

Visual data explorer and robust schema evolution tools



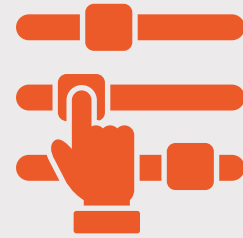
SETUP AND PROJECT STRUCTURE

INSTALLING PRISMA



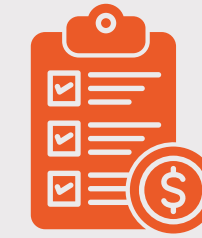
Install Prisma Packages

Add prisma (dev) and @prisma/client to your project



Initialize Prisma Setup

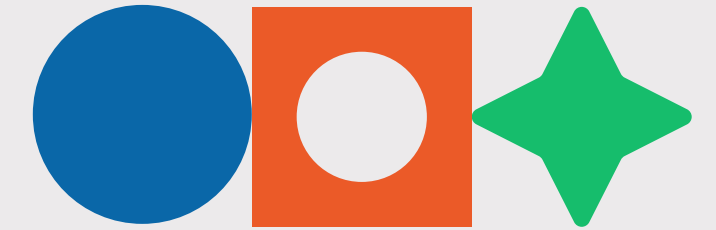
Run 'npx prisma init' to create config and schema files



Ready for Schema Definition

Start defining models in schema.prisma

PRISMA PROJECT STRUCTURE



Dedicated prisma/ Directory

Houses schema.prisma and all migration files

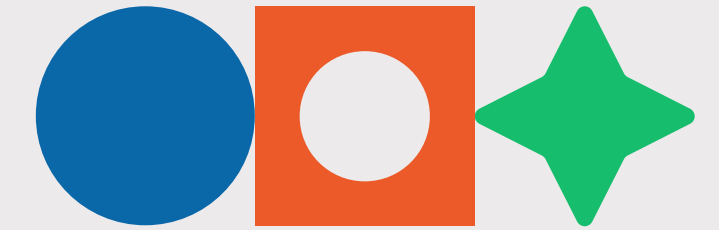
Separation of Concerns

Keeps database logic apart from application code

src/ for Application Logic

All business logic and API code lives here

PRISMA SCHEMA FILE



Centralized Schema Definition

Schema file specifies generators and datasources

Generator Block

Configures Prisma Client code generation

Datasource Block

Defines database type and connection details

Environment Variable Integration

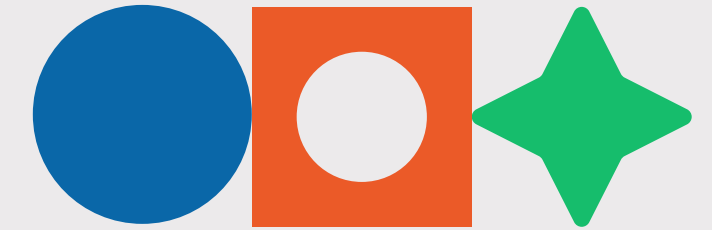
Database URL loaded securely from .env file





SCHEMA AND MODELS

DEFINING A MODEL



Models Mirror Database Tables

Each model defines a table structure in your database.

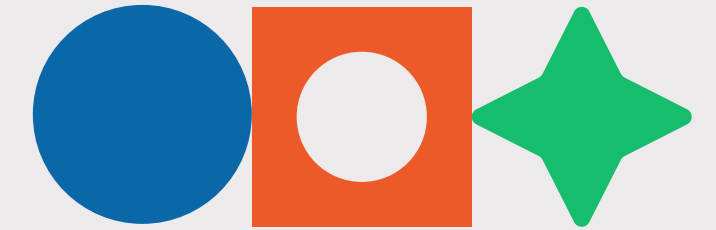
Explicit Field Types and Constraints

Specify data types, required fields, and relationships.

Built-in Defaults and Uniqueness

Set default values and enforce unique constraints easily.

PRISMA = DATABASE + TYPES



Unified Schema for DB and Types

Prisma schema defines both database tables and TypeScript types

No Manual Interfaces Needed

Type-safe Prisma Client auto-generates all required types

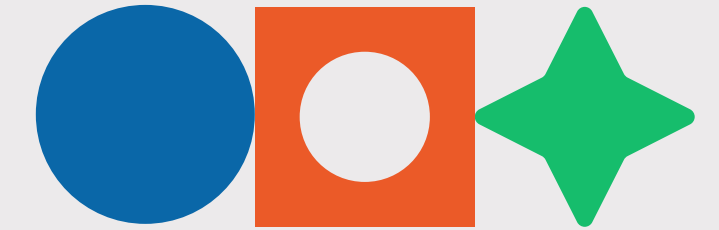
Compile-Time Query Safety

Catch type errors in queries before running your code



MIGRATIONS AND CLIENT USAGE

RUNNING MIGRATIONS



Create and Apply Migrations

Use `prisma migrate` to generate and run migration files

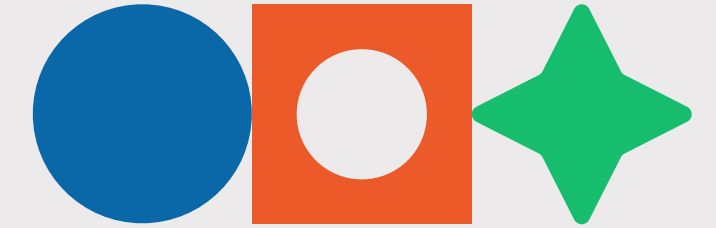
Keep Database and Schema in Sync

Migrations ensure your database matches your schema

Automatic Prisma Client Updates

Prisma Client is regenerated after each migration

PRISMA CLIENT USAGE



Import PrismaClient

Bring PrismaClient into your Node.js project

Instantiate a Singleton Client

Create one PrismaClient instance for app-wide use

Central Hub for Database Operations

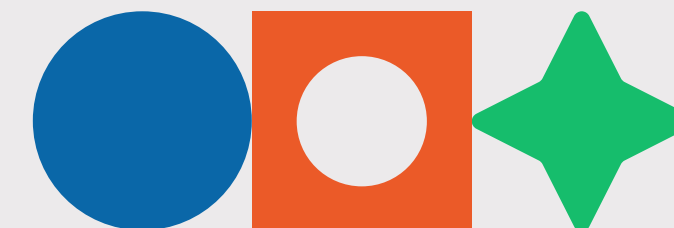
Use the client for all queries, inserts, updates, and deletes





CRUD OPERATIONS

CREATING DATA



Insert Records with `prisma.<model>.create`

Use the create method to add new entries to your database.

Pass Data as an Object

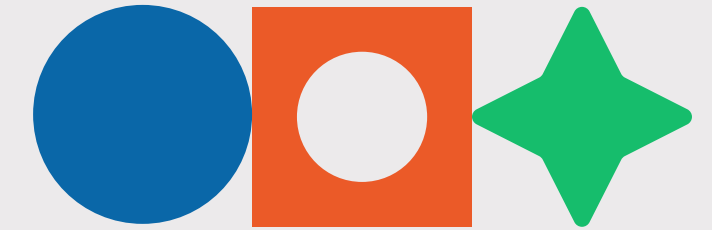
Provide field values in a structured object for clarity and type safety.

Type-Safe and Intuitive

Prisma ensures compile-time checks for your data structure.

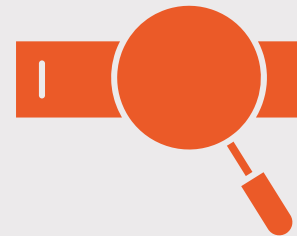


FETCHING DATA



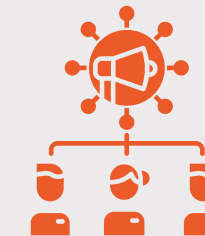
Retrieve Multiple Records Easily

Use `findMany` to fetch lists of records from a table.



Fetch a Single Record by Unique Field

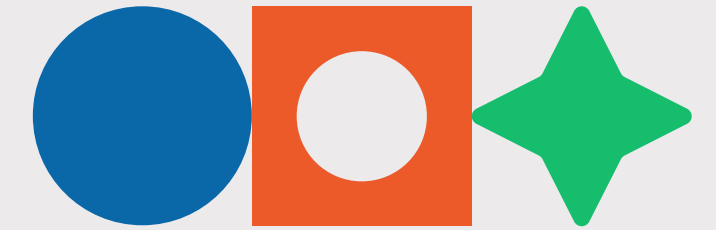
`findUnique` locates one record using a unique identifier.



Type-Safe Query Results

Prisma ensures returned data matches your model types.

UPDATING DATA



Update Records with update Method

Use `prisma.<model>.update` to change existing entries

Target Specific Records

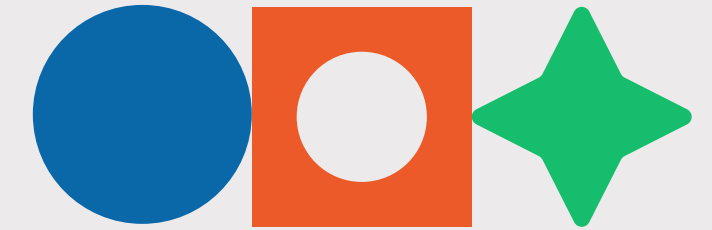
Define which record to update using the where clause

Set New Field Values

Provide updated data in the data object



DELETING DATA



Remove Records with delete Method

Effortlessly delete entries from your database

Target Specific Entries Using where

Precisely specify which record to remove

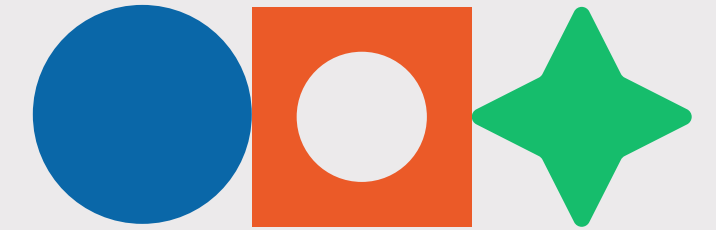
Ensures Data Integrity

Prevents accidental mass deletions



ADVANCED FEATURES

RELATIONS IN PRISMA



Model Relationships Made Simple

Define how tables connect using model fields

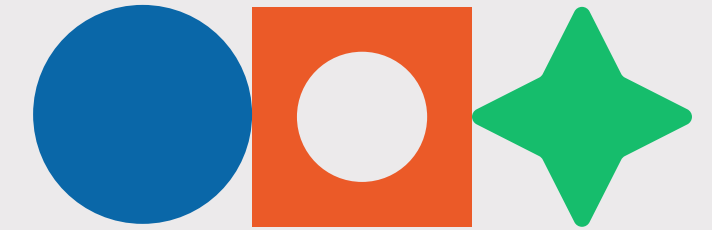
Foreign Keys with @relation

Use @relation to link models via foreign keys

Supports One-to-Many & Many-to-One

Easily model parent-child and reverse associations

FETCHING RELATIONS



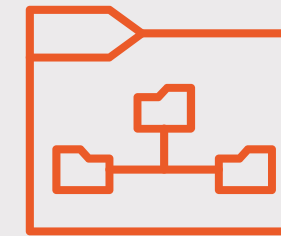
Include Related Models in Queries

Use the 'include' option to fetch associated data



Eager Loading of Relations

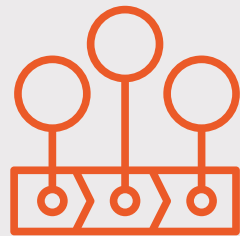
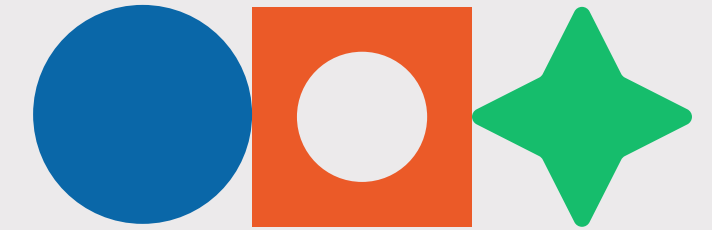
Retrieve parent and related records in a single query



Simplifies Data Retrieval

Reduces need for manual joins or multiple queries

PAGINATION



Efficient Pagination with skip & take

Retrieve specific data slices using skip and take options



Scalable Data Handling

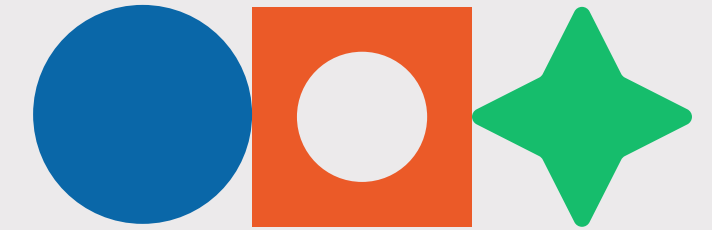
Process large datasets without overloading memory



Improved User Experience

Load data in manageable chunks for faster response times

FILTERING & SORTING



Flexible Filtering with `where`

Use `where` to select records matching specific criteria



Sorting Results with `orderBy`

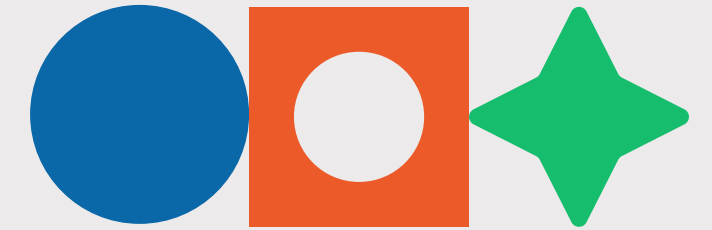
Apply `orderBy` to sort query results by any field



Combine Filtering and Sorting

Chain `where` and `orderBy` for precise, ordered data retrieval

TRANSACTIONS



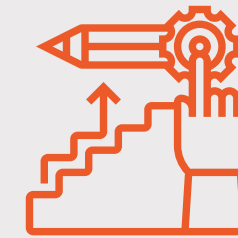
Atomic Multi-Step Operations

Bundle several database actions into a single transaction.



Consistency with \$transaction

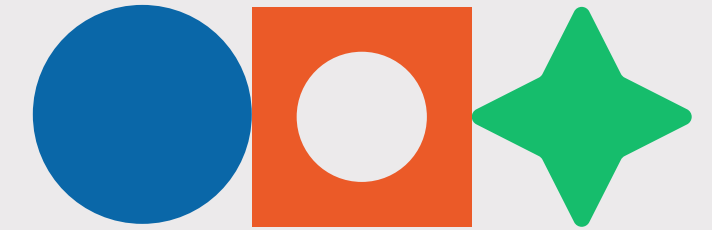
Use `prisma.$transaction` to ensure all-or-nothing execution.



Rollback on Failure

If any step fails, all changes are reverted automatically.

RAW QUERIES (WHEN NEEDED)



Direct SQL Execution with \$queryRaw

Run custom SQL queries when ORM features fall short

Reserved for Advanced Use Cases

Ideal for complex queries or performance tuning

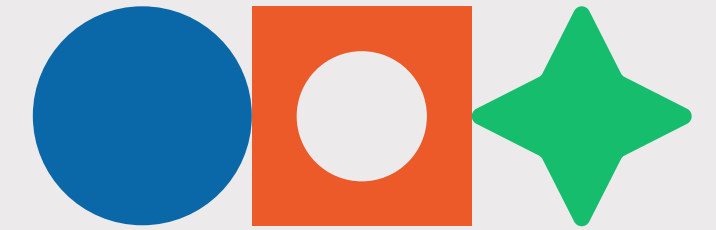
Caution: Use Sparingly

Bypass type safety and abstraction layers



TOOLS AND BEST PRACTICES

PRISMA STUDIO



Intuitive Visual Database Browser

Explore tables and records with a user-friendly interface

Edit and Inspect Data Instantly

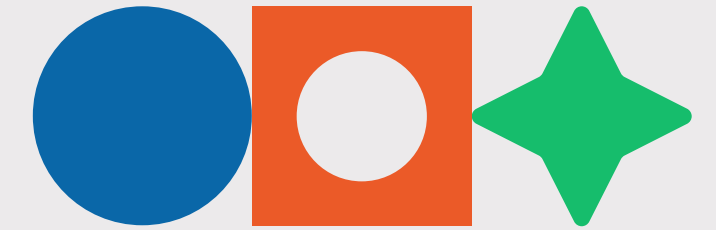
Modify, add, or delete records directly in your browser

Debug Schema and Relationships

Visually verify model structures and data integrity



BEST PRACTICES



Use a Singleton Prisma Client

Prevents connection issues and resource leaks

Always Apply Migrations

Keep schema and database in sync with migrations

Leverage Model Relations

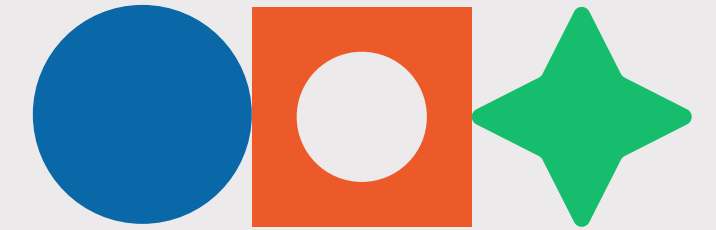
Define relations in schema instead of manual joins

Limit Raw SQL Usage

Use raw queries only for advanced scenarios



COMMON PITFALLS



Avoid Persistent Client Connections

Release PrismaClient promptly to prevent resource leaks.

Limit Use of 'include' in Queries

Fetching too many relations can slow down performance.

Don't Treat Prisma as a Query Builder

Leverage Prisma's ORM features, not just raw SQL capabilities.

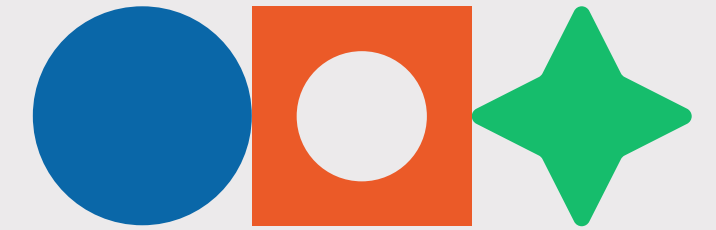
Always Run Migrations

Keep your database schema and Prisma schema in sync.



USE CASES AND COMPARISONS

WHEN TO USE PRISMA



Best for TypeScript-First Projects

Leverages type safety and modern tooling

Ideal for Rapid Development & SaaS

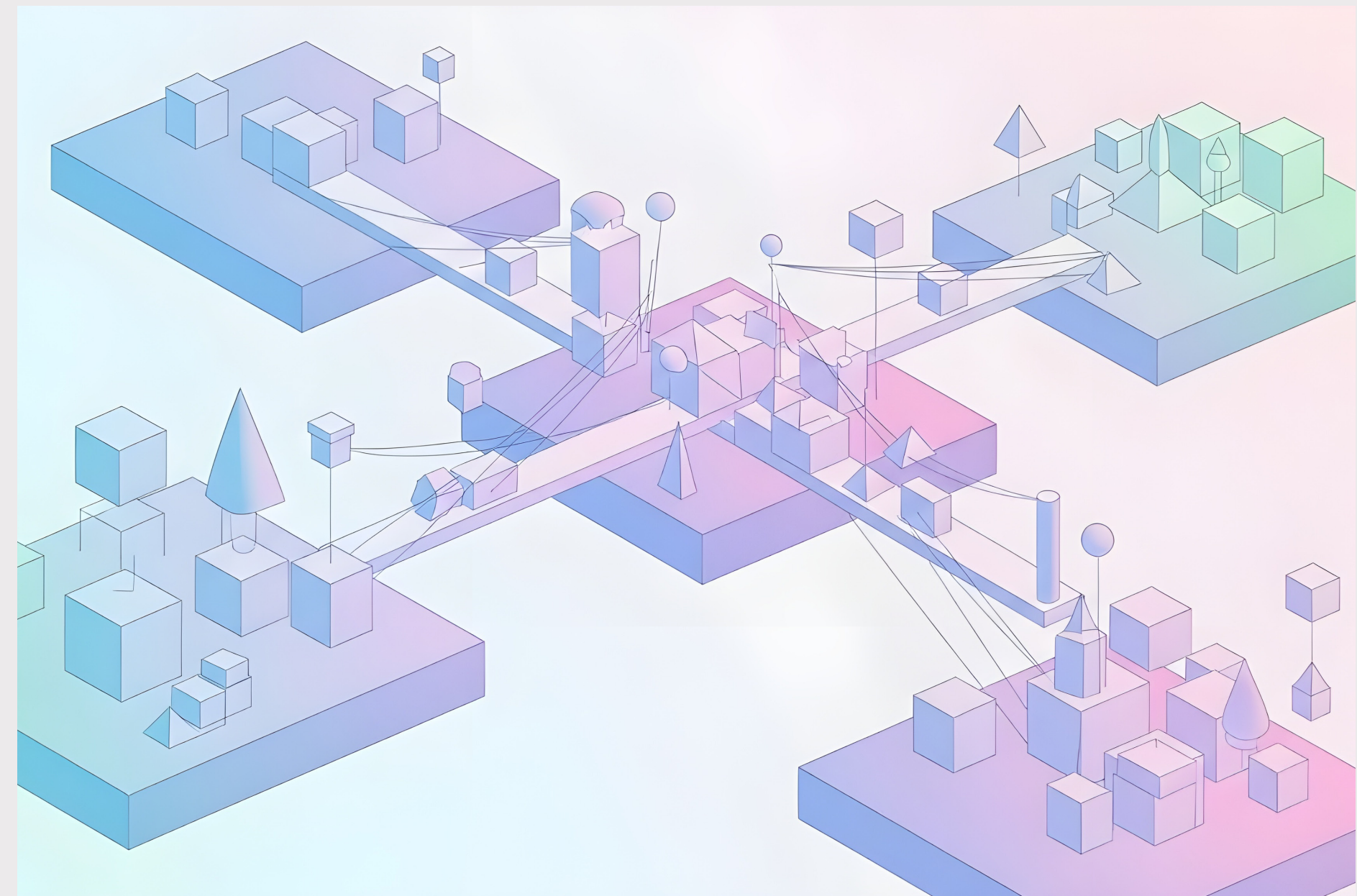
Accelerates prototyping and production for startups

Not Suited for Complex Legacy SQL

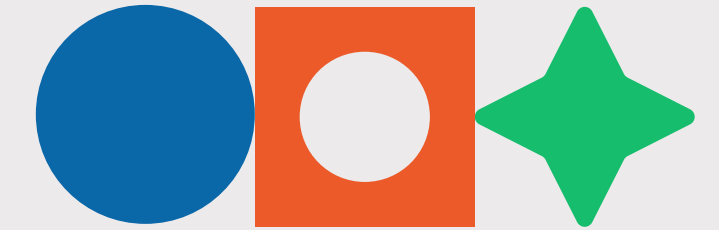
Limited flexibility for intricate, legacy database logic

Avoid if Heavy Stored Procedures Needed

Prisma is not designed for stored procedure-heavy workflows



PRISMA VS TYPEORM (QUICK VIEW)



Prisma: Unmatched Type Safety

Auto-generates types for compile-time query validation

TypeORM: Greater SQL Flexibility

Offers more direct control over SQL and legacy features

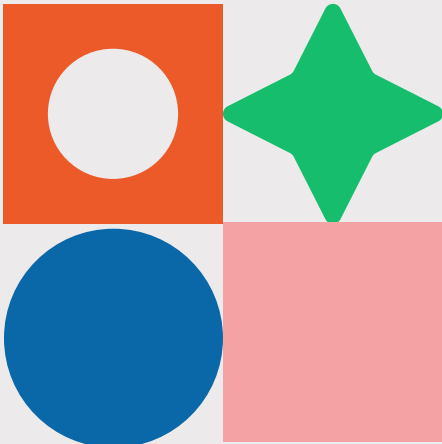
Developer Experience (DX)

Prisma provides a modern, intuitive workflow

NestJS Integration

Both ORMs integrate smoothly with NestJS projects

FINAL TAKEAWAY



Benefit	Description
Speed	Rapid development and efficient queries
Safety	End-to-end type safety and compile-time checks
Developer Happiness	Intuitive API and excellent developer experience
Production-Ready	Robust, reliable, and easy to use in real-world apps